DATABASE NORMALIZATION

1NF • 2NF • 3NF

Step-by-Step Examples, Design Decisions & Teaching Guide

First Normal Form (1NF)	Second Normal Form (2NF)	Third Normal Form (3NF)
Atomicity & Primary Keys	Full Functional Dependency	No Transitive Dependency

How to Use This Document

- Each Normal Form builds on the previous — read them in order.
- Every section includes a Problem Table, identified anomalies, step-by-step decomposition, and the resulting normalized tables.
- Design Decision boxes explain WHY we make each choice — critical for deep understanding.
- Multiple worked examples reinforce patterns so students can recognize them independently.
- Quick Summary cards at the end of each section work as handy revision aids.

0. Foundations: Why Normalization Matters

Normalization is the process of organizing a relational database to reduce data redundancy and improve data integrity. Edgar F. Codd introduced the concept in 1970. Without normalization, databases suffer from three classic problems known as

Update Anomalies.

Anomaly	What Goes Wrong	Real-World Example
Update Anomaly	Changing one fact requires updating multiple rows — miss one and the data becomes inconsistent.	A student changes address. If the address appears in 10 enrollment rows, all 10 must be updated. One missed = corrupt data.
Insert Anomaly	You cannot add new data without also supplying unrelated data.	You cannot store a new course unless at least one student is enrolled — the primary key requires StudentID.
Delete Anomaly	Deleting one piece of data inadvertently destroys other unrelated data.	The last student withdraws from a course. Deleting that row erases the course's existence from the database entirely.

Key Terminology Reference

Term	Definition
Primary Key (PK)	One or more columns that uniquely identify every row in a table. Cannot be NULL.
Composite Key	A primary key made of two or more columns together — neither column alone is unique enough.
Foreign Key (FK)	A column that references the Primary Key of another table, creating a relationship link.
Functional Dependency (FD)	Column B is functionally dependent on column A if knowing A's value always tells you B's value. Written: A → B
Partial Dependency	A non-key column depends on only part of a composite primary key (not the whole key). Violates 2NF.
Transitive Dependency	A non-key column depends on another non-key column, which in turn depends on the key. Violates 3NF.
Candidate Key	Any column (or set) that could serve as a primary key — unique and non-null.
Determinant	The left-hand side of a functional dependency. A determines B means A is the determinant.
Decomposition	Splitting one table into two or more tables to eliminate anomalies.
Lossless Join	A decomposition where the original table can be perfectly reconstructed by joining the results.

The Normalization Roadmap

Normal forms are cumulative. A table in 3NF automatically satisfies 2NF and 1NF. Think of them as levels:

Level	Rule	Eliminates
1NF	Atomic values + no repeating groups + primary key	Repeating groups, multi-valued cells
2NF	1NF + no partial dependencies on composite key	Partial functional dependencies
3NF	2NF + no transitive dependencies	Transitive functional dependencies

1. First Normal Form (1NF)

Definition of 1NF

A table is in First Normal Form (1NF) if ALL of the following conditions are true:

- Every column contains ATOMIC (indivisible) values — no lists, no sets, no multi-value cells.
- Every column contains values of the SAME TYPE (no mixing data types in a column).
- Each column has a UNIQUE NAME.
- The ORDER of rows does NOT matter (a relation, not a list).
- There is a PRIMARY KEY — a column (or combination) that uniquely identifies each row.

1.1 Example 1 — Student Enrollments (Multi-value cells)

Problem Table (NOT in 1NF)

Scenario: A school stores student enrollment data. The database administrator placed multiple values in single cells to save space.

StudentID	StudentName	Courses Enrolled	Grades
S001	Priya Sharma	Math, Physics, Chemistry	85, 90, 78
S002	Arjun Mehta	Biology, English	92, 88
S003	Sneha Patel	Math, Biology, History, Art	75, 80, 85, 91
S004	Rahul Gupta	Physics	95

⚠ Violations Identified

Why this table FAILS 1NF:

- **"Math, Physics, Chemistry"** — multiple values crammed into one cell (the Courses column). This is a repeating group.
- **"85, 90, 78"** — the Grades column mirrors the courses list. You cannot query "What grade did Priya get in Physics?" directly.
- **No clear Primary Key** — StudentID alone doesn't uniquely identify each enrollment record once we expand correctly.

Anomalies caused:

- UPDATE: Changing Priya's Chemistry grade requires parsing a string and rebuilding it.
- INSERT: Adding a new course to Sneha requires appending to a string — fragile and error-prone.
- QUERY: SQL cannot easily filter, group, or sort by values inside comma-separated strings.

Step-by-Step Conversion to 1NF

STEP 1 Expand multi-value cells — one row per atomic value

Each (Student, Course, Grade) combination gets its own row. We do NOT store lists in cells.

StudentID	StudentName	Course	Grade
S001	Priya Sharma	Math	85
S001	Priya Sharma	Physics	90
S001	Priya Sharma	Chemistry	78
S002	Arjun Mehta	Biology	92
S002	Arjun Mehta	English	88
S003	Sneha Patel	Math	75
S003	Sneha Patel	Biology	80
S003	Sneha Patel	History	85
S003	Sneha Patel	Art	91
S004	Rahul Gupta	Physics	95

STEP 2 **Identify the Primary Key**

StudentID alone is no longer unique (S001 appears 3 times). The unique identifier for each row is the combination of **(StudentID, Course)** — a Composite Primary Key.

Result: 1NF Table — Student_Enrollments

- **Primary Key:** (StudentID, Course) — composite
- Every cell holds exactly one value (atomic).
- Every column has a unique name.
- Row order does not convey meaning.
- SQL queries can now filter: WHERE Course = 'Physics'

Design Decision: Why Composite Keys?

When no single column uniquely identifies a row, we use a Composite Primary Key. Here, one student can take many courses, and one course is taken by many students — a many-to-many relationship. The only way to identify a specific enrollment is with both pieces of information together.

Alternative design: Add an auto-increment EnrollmentID column as a surrogate key. This is valid but introduces an artificial identifier. The composite key is more semantically meaningful and prevents duplicate enrollments at the database level.

1.2 Example 2 — Product Orders (Repeating Column Groups)

Problem Table (NOT in 1NF)

Scenario: An e-commerce company stores orders with multiple items per order using repeating column groups (Item1, Item2, Item3...).

OrderID	CustomerName	Item1	Qty1	Price1	Item2	Qty2	Price2	Item3	Qty3	Price3
O001	Amita Singh	Laptop	1	₹55000	Mouse	2	₹800			
O002	Dev Nair	Phone	1	₹28000	Cover	1	₹500	Charger	1	₹1200
O003	Ritu Shah	Tablet	2	₹18000						

⚠ Violations Identified

- **Repeating column groups:** Item1/Qty1/Price1, Item2/Qty2/Price2, Item3/Qty3/Price3 — same attribute repeated with numbered suffixes.
- **Wasted space:** Orders with fewer than 3 items have NULL columns. This wastes storage and complicates queries.
- **Scalability failure:** What if an order has 10 items? You'd need 10 sets of columns. The schema cannot adapt without a structural change.
- **Query complexity:** "Find all orders containing a Laptop" requires: WHERE Item1='Laptop' OR Item2='Laptop' OR Item3='Laptop'

Conversion to 1NF

STEP 1 **Eliminate repeating column groups**

Each item gets its own row. We introduce an **ItemLineNo** to distinguish multiple items within the same order.

OrderID	CustomerName	ItemLineNo	Item	Qty	UnitPrice
O001	Amita Singh	1	Laptop	1	₹55000
O001	Amita Singh	2	Mouse	2	₹800
O002	Dev Nair	1	Phone	1	₹28000
O002	Dev Nair	2	Cover	1	₹500
O002	Dev Nair	3	Charger	1	₹1200
O003	Ritu Shah	1	Tablet	2	₹18000

☑ Result: 1NF — Orders table

- **Primary Key: (OrderID, ItemLineNo)**
- No repeating columns — schema is fixed and clean.
- Any number of items per order without schema changes.
- Simple query: WHERE Item = 'Laptop'

1.3 Quick 1NF Checklist

Check	Fix If Failing

✓	All cells contain a single value?	Expand multi-value cells into multiple rows
✓	No comma-separated lists in any column?	Split lists into separate rows with repeated key
✓	No repeating column groups (Item1, Item2...)?	Convert to rows with a sequence number column
✓	Each column holds one data type consistently?	Separate mixed-type columns or validate data
✓	A Primary Key exists (single or composite)?	Identify the minimal set of columns that guarantees uniqueness

2. Second Normal Form (2NF)

Definition of 2NF

A table is in Second Normal Form (2NF) if:

- **It is already in 1NF, AND**
- **Every non-key attribute is FULLY functionally dependent on the ENTIRE Primary Key.**

In plain English: if you have a composite primary key (PK made of multiple columns), EVERY other column must depend on ALL of those key columns together — not just one part of the key.

⚠ Note: If a table has a single-column primary key, it is automatically in 2NF (partial dependency is impossible with a single-column key).

Understanding Functional Dependencies

Before identifying 2NF violations, you must be able to identify Functional Dependencies (FDs). An FD written as **A → B** means: knowing the value of A always tells you the value of B.

Worked Example of FD Notation

- **StudentID → StudentName** (Knowing the StudentID tells you the student's name)
- **CourseID → CourseName** (Knowing the CourseID tells you the course name)
- **(StudentID, CourseID) → Grade** (You need BOTH to find the grade)
- **StudentID → StudentName is a PARTIAL dependency** if PK = (StudentID, CourseID), because Grade depends only on PART of the key.

2.1 Example 1 — Student Enrollments (continuing from 1NF)

Table After 1NF (NOT yet in 2NF)

Our 1NF table from Section 1. The Primary Key is **(StudentID, CourseID)**. Let us add more detail to make the problem clearer:

StudentID (PK)	CourseID (PK)	StudentName	StudentEmail	CourseName	Credits	Grade
S001	C101	Priya Sharma	priya@college.edu	Mathematics	4	85
S001	C102	Priya Sharma	priya@college.edu	Physics	3	90
S001	C103	Priya Sharma	priya@college.edu	Chemistry	3	78
S002	C101	Arjun Mehta	arjun@college.edu	Mathematics	4	88
S002	C104	Arjun Mehta	arjun@college.edu	Biology	4	92
S003	C102	Sneha Patel	sneha@college.edu	Physics	3	75

StudentID (PK)	CourseID (PK)	StudentName	StudentEmail	CourseName	Credits	Grade
S003	C101	Sneha Patel	sneha@college.edu	Mathematics	4	82

Functional Dependency Analysis

Functional Dependency	Determinant	Dependent	Type
StudentID → StudentName	StudentID only	StudentName	PARTIAL ✗ — Violation!
StudentID → StudentEmail	StudentID only	StudentEmail	PARTIAL ✗ — Violation!
CourseID → CourseName	CourseID only	CourseName	PARTIAL ✗ — Violation!
CourseID → Credits	CourseID only	Credits	PARTIAL ✗ — Violation!
(StudentID, CourseID) → Grade	Full composite key	Grade	FULL ✓ — Correct

⚠ Problems Caused by Partial Dependencies

- **UPDATE Anomaly:** Priya Sharma changes email → must update 3 rows. Miss one = inconsistent data.
- **INSERT Anomaly:** Cannot add a new course (C105 - History) to the database unless some student is enrolled.
- **DELETE Anomaly:** If Sneha drops Math (her last course), we lose Sneha's contact information entirely.
- **Redundancy:** "Priya Sharma", "priya@college.edu" is stored once per course — completely unnecessary repetition.

Step-by-Step Decomposition to 2NF

STEP 1 Group columns by what determines them

Separate the columns based on which part of the key they depend on:

Depends on StudentID only	Depends on CourseID only	Depends on BOTH (Full FD)
StudentID, StudentName, StudentEmail	CourseID, CourseName, Credits	(StudentID, CourseID), Grade

STEP 2 Create separate tables for each group

Table 1: Students

StudentID (PK)	StudentName	StudentEmail
S001	Priya Sharma	priya@college.edu
S002	Arjun Mehta	arjun@college.edu
S003	Sneha Patel	sneha@college.edu

Table 2: Courses

CourseID (PK)	CourseName	Credits
C101	Mathematics	4
C102	Physics	3
C103	Chemistry	3
C104	Biology	4

Table 3: Enrollments (junction/fact table)

StudentID (PK, FK)	CourseID (PK, FK)	Grade
S001	C101	85
S001	C102	90
S001	C103	78
S002	C101	88
S002	C104	92
S003	C102	75
S003	C101	82

☑ 2NF Achieved — Benefits

- No redundancy: Priya's name/email stored once, regardless of how many courses she takes.
- No insert anomaly: New courses can be added to the Courses table independently.
- No delete anomaly: Dropping all enrollments doesn't erase student or course data.
- Grade is the only column in Enrollments — it genuinely needs BOTH keys to be identified.

Design Decision: The Junction Table Pattern

When two entities have a many-to-many relationship (a student can enroll in many courses; a course can have many students), you MUST use a junction (bridge/associative) table. This is a core pattern in relational database design.

The junction table's composite primary key is formed from the foreign keys of both parent tables. Any attributes that describe the relationship itself (like Grade — which belongs to a specific enrollment, not to the student or course alone) belong in this junction table.

Tip for students: If you can ask 'what is the [attribute] of [Entity A] AND [Entity B]?', that attribute belongs in the junction table.

2.2 Example 2 — Employee Projects

Problem Table (in 1NF but NOT 2NF)

Scenario: A company tracks which employees work on which projects. Primary Key: **(EmployeeID, ProjectID)**

EmpID (PK)	ProjectID (PK)	EmpName	EmpDept	ProjectName	ProjectBudget	HoursWorked
E01	P001	Kavya Reddy	Engineering	Alpha Platform	₹500000	120
E01	P002	Kavya Reddy	Engineering	Beta Mobile	₹300000	80
E02	P001	Rajan Iyer	Marketing	Alpha Platform	₹500000	40
E03	P002	Meera Das	Design	Beta Mobile	₹300000	200
E03	P003	Meera Das	Design	Gamma Web	₹150000	60

FD Analysis

Partial dependencies are highlighted:

FD Identification

- **EmpID → EmpName, EmpDept** ← PARTIAL (only depends on EmpID, not ProjectID)
- **ProjectID → ProjectName, ProjectBudget** ← PARTIAL (only depends on ProjectID, not EmpID)
- **(EmpID, ProjectID) → HoursWorked** ← FULL (how many hours this person worked on this project)

Decomposed 2NF Tables

Table 1: Employees

EmpID (PK)	EmpName	EmpDept
E01	Kavya Reddy	Engineering
E02	Rajan Iyer	Marketing
E03	Meera Das	Design

Table 2: Projects

ProjectID (PK)	ProjectName	ProjectBudget
P001	Alpha Platform	₹500000
P002	Beta Mobile	₹300000

ProjectID (PK)	ProjectName	ProjectBudget
P003	Gamma Web	₹150000

Table 3: ProjectAssignments

EmpID (PK, FK)	ProjectID (PK, FK)	HoursWorked
E01	P001	120
E01	P002	80
E02	P001	40
E03	P002	200
E03	P003	60

3. Third Normal Form (3NF)

Definition of 3NF

A table is in Third Normal Form (3NF) if:

- **It is already in 2NF, AND**
- **No non-key attribute is transitively dependent on the primary key.**

Transitive dependency: A → B → C, where A is the PK, B and C are non-key attributes. If A determines B, and B determines C, then C is transitively dependent on A.

In plain English: every non-key column must tell you something about the key, the whole key, and NOTHING BUT the key.

Understanding Transitive Dependencies

A transitive dependency forms a chain. Consider this 2NF table:

StudentID (PK)	StudentName	ZipCode	City	State
S001	Priya Sharma	380015	Ahmedabad	Gujarat
S002	Arjun Mehta	400001	Mumbai	Maharashtra
S003	Sneha Patel	380015	Ahmedabad	Gujarat
S004	Rahul Gupta	110001	New Delhi	Delhi

Transitive Chain Identified

- **StudentID → ZipCode** (Each student has one zip code)
- **ZipCode → City** (A zip code always maps to the same city)
- **ZipCode → State** (A zip code always maps to the same state)

Therefore: StudentID → ZipCode → City, State ← This is a TRANSITIVE dependency!

City and State are not directly about the student. They are about the ZipCode. The student merely happens to live there.

Anomalies from Transitive Dependencies

- **UPDATE Anomaly:** If zip code 380015 moves to a different city, we must update ALL student rows with that zip — miss one and data is corrupt.
- **INSERT Anomaly:** Cannot store a new zip code and its city/state unless a student happens to live there.
- **DELETE Anomaly:** If the last student with zip 110001 is deleted, we lose the knowledge that 110001 is in New Delhi, Delhi.

Decomposition to 3NF

STEP 1 Identify all non-key → non-key dependencies (transitive chains)

STEP 2 Create a new table for each transitive determinant

Table 1: Students (3NF)

StudentID (PK)	StudentName	ZipCode (FK)
S001	Priya Sharma	380015
S002	Arjun Mehta	400001
S003	Sneha Patel	380015
S004	Rahul Gupta	110001

Table 2: ZipCodes (new — extracted from transitive chain)

ZipCode (PK)	City	State
380015	Ahmedabad	Gujarat
400001	Mumbai	Maharashtra
110001	New Delhi	Delhi

3NF Achieved — Benefits

- City and State for a zip code are stored exactly once — in the ZipCodes table.
- Updating a city name requires changing exactly one row in ZipCodes.
- New zip codes can be added to ZipCodes independently.
- Every non-key column in Students tells us something directly about the student.

Design Decision: When to Stop Normalizing?

Real-world databases often go up to 3NF and stop. Higher normal forms (BCNF, 4NF, 5NF) exist but have diminishing returns for most applications. Over-normalization can hurt performance — retrieving data requires more joins, which are expensive.

Guiding principle: Normalize until it hurts, then denormalize until it works. For OLTP (transactional) systems, 3NF is the standard target. For OLAP (analytical/reporting) systems, some denormalization (star schemas) is often preferred for query speed.

Always ask: Does this extra join save us from real anomalies? If the answer is no, the normalization may not be worth the complexity.

3.1 Example 2 — Hospital Appointments

Problem Table (2NF but NOT 3NF)

Scenario: A hospital tracks patient appointments. The table is already in 2NF (single-column primary key — no partial dependencies possible). We now check for transitive dependencies.

Appt ID (PK)	PatientName	DoctorID	DoctorName	DoctorSpecialty	DeptID	DeptName	DeptFloor	ApptDate	Diagnosis
A001	Priya Sharma	D01	Dr. Mehta	Cardiology	DEPT01	Heart Centre	3rd	2024-01-15	Hypertension
A002	Arjun Nair	D02	Dr. Patel	Neurology	DEPT02	Brain & Spine	5th	2024-01-16	Migraine
A003	Sneha Das	D01	Dr. Mehta	Cardiology	DEPT01	Heart Centre	3rd	2024-01-16	Arrhythmia
A004	Rahul Singh	D03	Dr. Kumar	Orthopedics	DEPT03	Bone & Joint	2nd	2024-01-17	Fracture
A005	Meera Roy	D01	Dr. Mehta	Cardiology	DEPT01	Heart Centre	3rd	2024-01-18	Chest Pain

Finding All Transitive Dependencies

Transitive Chain	Problem	Solution
ApptID → DoctorID → DoctorName, DoctorSpecialty	Doctor info repeated for every appointment	Extract Doctors table
ApptID → DoctorID → DeptID → DeptName, DeptFloor	Dept info repeated per appointment AND per doctor	Extract Departments table
ApptID → PatientName	Patient name repeated across appointments	Extract Patients table with PatientID

Decomposed 3NF Tables

Table 1: Departments

DeptID (PK)	DeptName	DeptFloor
DEPT01	Heart Centre	3rd
DEPT02	Brain & Spine	5th
DEPT03	Bone & Joint	2nd

Table 2: Doctors

DoctorID (PK)	DoctorName	DoctorSpecialty	DeptID (FK)
D01	Dr. Mehta	Cardiology	DEPT01
D02	Dr. Patel	Neurology	DEPT02
D03	Dr. Kumar	Orthopedics	DEPT03

Table 3: Patients

PatientID (PK)	PatientName
P01	Priya Sharma
P02	Arjun Nair
P03	Sneha Das
P04	Rahul Singh
P05	Meera Roy

Table 4: Appointments (3NF — lean fact table)

ApptID (PK)	PatientID (FK)	DoctorID (FK)	ApptDate	Diagnosis
A001	P01	D01	2024-01-15	Hypertension
A002	P02	D02	2024-01-16	Migraine
A003	P03	D01	2024-01-16	Arrhythmia
A004	P04	D03	2024-01-17	Fracture
A005	P05	D01	2024-01-18	Chest Pain

☑ All 3NF Rules Verified for Appointments Table

- **ApptID → PatientID:** ✓ Direct — PatientID is a FK, tells us which patient.
- **ApptID → DoctorID:** ✓ Direct — DoctorID is a FK, tells us which doctor.
- **ApptID → ApptDate:** ✓ Direct — the date belongs to this appointment specifically.
- **ApptID → Diagnosis:** ✓ Direct — the diagnosis is specific to this appointment visit.
- **No transitive chains remain** — DoctorName, DeptName etc. are in their own tables.

4. Full Worked Example: Unnormalized → 1NF → 2NF → 3NF

This section takes a single messy table all the way from Unnormalized Form (UNF) to 3NF in one complete walkthrough — the most important exercise for exam preparation.

Scenario: Library Book Loans System

A small library has been storing all loan information in a single spreadsheet for years. The librarian has come to you and asked you to design a proper database. Below is a sample of their data.

4.1 The Unnormalized Table

LoanID	MemberName	MemberPhone	MemberCity	CityState	Books Borrowed (Title, Author, ISBN)	Loan Date	Due Date	Fine (₹)
L001	Anjali Verma	9876543210	Ahmedabad	Gujarat	The Alchemist (Paulo Coelho, 978-0-06), Wings of Fire (APJ Kalam, 978-8-17)	01-Jan-2024	15-Jan-2024	0, 25
L002	Vikram Nair	8765432109	Bangalore	Karnataka	Sapiens (Yuval Harari, 978-0-09)	05-Jan-2024	19-Jan-2024	0
L003	Anjali Verma	9876543210	Ahmedabad	Gujarat	1984 (George Orwell, 978-0-45)	10-Jan-2024	24-Jan-2024	50

4.2 Step 1: Convert to 1NF

1NF Violations Found

- Books Borrowed column contains multiple values (book+author+ISBN in one cell).
- Fine column contains multiple fine values for multiple books.
- No atomic ISBN, Author, or Title columns.

Fix: One row per book borrowed. Composite PK = (LoanID, ISBN).

Loan ID (PK)	ISBN (PK)	MemberName	MemberPhone	Member City	CitySt ate	BookT itle	BookAu thor	LoanD ate	DueD ate	Fine (₹)
L001	978-0-06	Anjali Verma	9876543210	Ahmedabad	Gujarat	The Alchemist	Paulo Coelho	01-Jan-24	15-Jan-24	0
L001	978-8-17	Anjali Verma	9876543210	Ahmedabad	Gujarat	Wings of Fire	APJ Kalam	01-Jan-24	15-Jan-24	25
L002	978-0-09	Vikram Nair	8765432109	Bangalore	Karnataka	Sapiens	Yuval Harari	05-Jan-24	19-Jan-24	0
L003	978-0-45	Anjali Verma	9876543210	Ahmedabad	Gujarat	1984	George Orwell	10-Jan-24	24-Jan-24	50

✓ 1NF Status

› Atomic values: each cell holds exactly one value.
› Primary Key: (LoanID, ISBN)
› No repeating groups or multi-value columns.

4.3 Step 2: Convert to 2NF

PK is composite: **(LoanID, ISBN)**. Check for partial dependencies:

Attribute	Depends On	Verdict
MemberName, MemberPhone	LoanID only	**PARTIAL ✗ (member doesn't depend on which book)**
MemberCity, CityState	LoanID only	**PARTIAL ✗**
BookTitle, BookAuthor	ISBN only	**PARTIAL ✗ (book info doesn't depend on LoanID)**
LoanDate, DueDate	LoanID only	**PARTIAL ✗ (loan dates belong to the loan, not the book)**
Fine(₹)	(LoanID, ISBN)	**FULL ✓ (fine is specific to this loan + this book)**

2NF Decomposition

Members table

LoanID (PK)	MemberName	MemberPhone	MemberCity	CityState
L001	Anjali Verma	9876543210	Ahmedabad	Gujarat
L002	Vikram Nair	8765432109	Bangalore	Karnataka

LoanID (PK)	MemberName	MemberPhone	MemberCity	CityState
L003	Anjali Verma	9876543210	Ahmedabad	Gujarat

Books table

ISBN (PK)	BookTitle	BookAuthor
978-0-06	The Alchemist	Paulo Coelho
978-8-17	Wings of Fire	APJ Kalam
978-0-09	Sapiens	Yuval Harari
978-0-45	1984	George Orwell

Loans table (fact table)

LoanID (PK)	ISBN (PK, FK)	LoanDate	DueDate	Fine(₹)
L001	978-0-06	01-Jan-2024	15-Jan-2024	0
L001	978-8-17	01-Jan-2024	15-Jan-2024	25
L002	978-0-09	05-Jan-2024	19-Jan-2024	0
L003	978-0-45	10-Jan-2024	24-Jan-2024	50

4.4 Step 3: Convert to 3NF

Now check for transitive dependencies in each 2NF table:

Transitive Dependencies Found

- **In Members: LoanID → MemberCity → CityState** ← CityState depends on MemberCity, not on LoanID directly.
- **In Members:** Anjali Verma (L001, L003) — MemberName and MemberPhone appear in two rows. MemberID is needed.

Final 3NF Tables

Cities table (new)

MemberCity (PK)	CityState
Ahmedabad	Gujarat
Bangalore	Karnataka

Members table (3NF — refined)

MemberID (PK)	MemberName	MemberPhone	MemberCity (FK)
M01	Anjali Verma	9876543210	Ahmedabad
M02	Vikram Nair	8765432109	Bangalore

Loans table (3NF — uses MemberID FK)

LoanID (PK)	MemberID (FK)	ISBN (PK, FK)	LoanDate	DueDate	Fine(₹)
L001	M01	978-0-06	01-Jan-2024	15-Jan-2024	0
L001	M01	978-8-17	01-Jan-2024	15-Jan-2024	25
L002	M02	978-0-09	05-Jan-2024	19-Jan-2024	0
L003	M01	978-0-45	10-Jan-2024	24-Jan-2024	50

Books table — already in 3NF (no changes needed)

ISBN → BookTitle and ISBN → BookAuthor. Both attributes describe the book directly. No transitive chain.

5. Quick Reference & Exam Preparation

5.1 The Golden Rules — All Three Normal Forms

NF	One-Line Rule	What to Fix	How to Fix
1NF	Atomic values + primary key	Multi-value cells, repeating groups, numbered column sets	Expand: one row per atomic value. Add composite PK.
2NF	Full dependency on whole key	Non-key column depends on part of composite PK	Move partial-dependent columns to a new table keyed by that partial key.
3NF	No non-key determines another non-key	A → B → C where B and C are non-key attributes	Extract B (the middle determinant) and its dependents into a new table.

5.2 Common Mistakes Students Make

#	Mistake	Correct Understanding
1	Thinking 2NF applies to single-column PKs.	2NF is ONLY relevant when the Primary Key is composite. A single-column PK cannot have partial dependencies by definition.
2	Confusing partial and transitive dependencies.	Partial = non-key depends on PART of the key. Transitive = non-key depends on ANOTHER non-key (which depends on the key).
3	Forgetting to keep FKs in the original table after decomposition.	When you move columns to a new table, leave a FK column behind in the original table to maintain the relationship.
4	Assuming normalization always makes the design better.	Over-normalization causes too many joins. For read-heavy or analytical workloads, some denormalization is deliberately chosen.
5	Thinking all redundancy is bad.	Controlled redundancy (FK values) is fine and necessary. Uncontrolled redundancy (same fact stored multiple times) is what normalization eliminates.
6	Not checking 2NF after achieving 1NF.	Each normal form must be verified in sequence. Reaching 1NF does not automatically give you 2NF or 3NF.

5.3 Practice Questions

Practice Exercise 1 — Identify the Normal Form

Given this table with PK = EmployeeID:

Employee(EmployeeID, EmpName, DeptID, DeptName, DeptLocation, ManagerID, ManagerName)

Identify: Is this table in 1NF? 2NF? 3NF?
Identify all functional dependencies.
Decompose to 3NF and name your resulting tables.

Answer hint: DeptID → DeptName, DeptLocation (transitive via DeptID). ManagerID → ManagerName (transitive via ManagerID). Result: Employees, Departments, Managers tables.

Practice Exercise 2 — Sales Data

Unnormalize this scenario into a single bad table, then normalize it step by step:

A company records: each salesperson (with their region and manager), can sell multiple products (each with a category and price) to multiple customers (with city and state) on different dates.

What columns would you put in the initial bad table? What violations appear at each level? What does the final 3NF schema look like?

Practice Exercise 3 — FD Identification

Given: Table T(A, B, C, D, E) where PK = (A, B)

FDs given: A → C, D | B → E | (A, B) → C, D, E

Questions:
1. Which FDs are partial dependencies?
2. Is this table in 2NF? Explain why or why not.
3. Decompose to 2NF. Name the resulting tables and their PKs.
4. Are the resulting tables in 3NF? Check for transitive dependencies.

Remember the Mantra

The key (1NF) — the whole key (2NF) — and nothing but the key (3NF)

So help me Codd.